AgentCore as the Sandbox in Agentic RL

Running slime's tool calls, checkers and agent loops on Amazon Bedrock AgentCore

Kaige Yang
Senior Applied Scientist, AWS

September 25, 2026

Agenda

01 Agentic RL

Architecture and pipeline: trainer, inference, harness, sandbox

02 AgentCore foundations

Runtime vs Code Interpreter, microVM vs Instance, session lifecycle, limits

03 Using AgentCore in slime

One hook, no fork; deploy, wire and network a Runtime

04 Demo: Terminal-Bench

Custom images at scale; pre-open and heartbeat hide the cold start

05 Demo: harness scaling (`demo_cpu_tasks SCALE=1`)

The harness tier, default in every demo, scaled with inference and the sandbox

06 Other demos

The same hook on other substrates

07 Conclusion

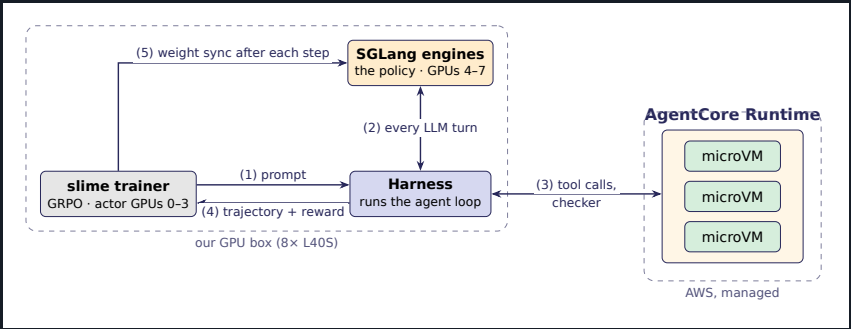
DeepSeek DSec vs AgentCore, and takeaways

Appendix (from slide 30): references.

© 2026, Amazon Web Services, Inc. or its affiliates.

The sandbox is the reward source: four tiers close one on-policy loop

Figure 1 **Agentic RL: trainer, inference, harness, sandbox**

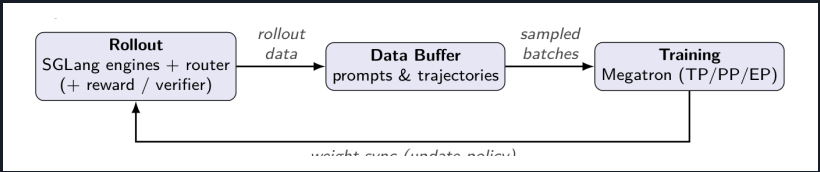


- **Trainer** (slime: Megatron, GRPO) updates the policy; **inference** (SGLang engines) serves it.
- **Harness** runs the agent loop; a proxy records the exact `token_ids` + logprobs (**TITO**), so training stays on-policy.
- **Sandbox** (AgentCore) runs the tool calls and the checker: **the sandbox is the reward source**.

© 2026, Amazon Web Services, Inc. or its affiliates.

Each episode becomes one loss-masked Sample, and rollout runs async with training

Figure 2 slime pipeline: rollout → data buffer → training → weight sync



1. **Seed:** the harness opens a sandbox session and writes the task's files.
2. **Act → observe:** policy turn → tool call runs in the sandbox → output fed back, for several turns.
3. **Check:** a hidden checker runs in the sandbox → binary reward (RLVR, no LLM judge).
4. **Pack:** one loss-masked `Sample` per episode → data buffer.
5. **Train + sync:** a GRPO step, weights pushed to SGLang; rollout runs **async** with training.

© 2026, Amazon Web Services, Inc. or its affiliates.

The same loop code runs on either substrate; only the Runtime ARN differs

- **Runtime:** your image, serverless, **per-session isolated compute**, called with `InvokeAgentRuntime` (SigV4).
- **Code Interpreter:** a managed CPU sandbox, no image to build.
- Same loop code on both substrates; only the Runtime ARN differs. **Both demos use microVMs.**

Table 1 **microVM vs Instance substrates**

	microVM (serverless)	**Instance** (EC2 via a capacity provider)
Compute	own guest kernel, **CPU only**	m5 (CPU) · g6 / g5 (**GPU**, whole cards, no MIG)
Image	**arm64**, $\leq$ ~**2 GB**	**amd64**, instance disk
Session length	$\leq$ 8 h	$\leq$ 14 days

Source: AgentCore substrate limits as stated in the project deck (`docs/slides/agentcore_slime_deck.md`) and report (`docs/report/agentcore_slime_report.md`); checked 2026-09-25.

© 2026, Amazon Web Services, Inc. or its affiliates.

A cold launch costs 0.77 s to minutes; a reused or pre-opened session opens in milliseconds

- **1 : N : 1.** Runtime = one image, one ARN; session = one `runtimeSessionId` = one microVM / instance. **Runtimes track images, not tasks.**
- **Launch:** no “create session” API; the first invoke under a new id cold-starts the compute.
- **End:** idle timeout, max lifetime, or `StopRuntimeSession`.

Table 2 **Measured launch cost by substrate**

Substrate	Cold launch (measured)	Reused open
Code Interpreter	p50 **1.83 s**, **every episode**	–
Own Runtime, microVM	≈ **0.77 s** (stdlib) · ≈ **15 s** (447 MB)	pre-open wait **5 ms**
Instance `m5` · `g6`	≈ **60–80 s** · ≈ **123–207 s**	0.12–0.33 s

Source: Report `docs/report/agentcore_slime_report.md`: `DATASET=libs` smoke (`start_session` p50, 2026-09-25); warm-start probe `warmstart_20260925_100103` (2026-09-25); GPU sweep (2026-09-21).

© 2026, Amazon Web Services, Inc. or its affiliates.

One RL step of 16 prompts $\times$ 8 rollouts needs 16 Runtimes and 128 fresh microVMs

Batch: 16 prompts, **each needs its own image**; GRPO samples **8 rollouts per prompt**.

Table 3 One batch mapped onto Runtimes, sessions and microVMs

RL concept	AgentCore object	Count for this batch
Prompt env	1 image $\rightarrow$ 1 **Runtime** (ARN)	**16 Runtimes**, deployed once, reused every step
Rollout	1 **session** (session id)	16 $\times$ 8 = **128 sessions**, new every step
Its compute	1 **microVM** per session	**128 microVMs** in flight, stopped at episode end
GRPO group	8 sessions on **one** Runtime	8 separate microVMs, nothing shared

- **AgentCore never sees prompts or rewards**, only an ARN + session id; the harness maps them.
- **Limits:** 128 opens at 25 /s $\geq$ **5 s**/step; 128 $\ll$ 5000 concurrent; Runtimes = **distinct images in the dataset** ($\leq$ 1000).
- **Cost:** 128 cold boots per step (0.8–15 s each) unless pre-opened (slide 16).

Source: Worked example; quotas from AgentCore Service Quotas (us-west-2, 2026-09-26), boot times from the warm-start probe (2026-09-25).

© 2026, Amazon Web Services, Inc. or its affiliates.

Silent idle reclaim, the 25/s open rate and the 1000-Runtime cap shape the design

Table 4 **AgentCore limits and how we handle them**

Limit	Value	What we do
Idle timeout	15 min default (we set 3600 s)	reclaim is **silent** (empty sandbox) $\rightarrow$ heartbeat
Max lifetime	microVM **8 h** · Instance 14 d	re-seed long episodes
New sessions	**25 /s** (adj.)	caps the cold-start burst $\rightarrow$ pre-open ahead
Concurrent sessions	**5000** (adj.)	one per in-flight episode
Code Interpreter	**30 /s** start and invoke	$\approx$ 75 episodes in flight
Runtimes per account	**1000**; create $\leq$ 50 /s	$\leq$ ~1000 custom images (Terminal-Bench ~229)
Image	microVM **2 GB**, arm64	bigger $\rightarrow$ slim it or use an Instance
Per request	**15 min** · 100 MB	long `exec` runs async, polled
Privileges	**no Linux capabilities**	no per-episode uid / cgroup inside one session

Source: AgentCore Service Quotas, this account, us-west-2 (2026-09-26); project deck `agentcore_slime_deck.md` (2026-09-25).

© 2026, Amazon Web Services, Inc. or its affiliates.

slime needs only its two existing hooks to use AgentCore, so there is no fork

`--custom-generate-function-path` → `generate.generate` (dispatches the episode, records turns); `--custom-rm-path` → the reward.

```
async def generate(args, sample, sampling_params): # slime hook
    task, pool = task_from(sample), harness_pool.get_pool(args)
    async def episode(worker): # a free dispatch worker pulls it
        async with pool.sandbox(key, open_session) as (sb, wait, slot): # sandbox first
            with pool.engine() as (e, engine): # then the least-loaded engine
                return await mini_claude_episode.run(..., sb, engine) # loop runs IN sb
    return await pool.run(episode) # ONE loss-masked Sample
```

- Each sandbox op = one `invoke_agent_runtime(ARN, session_id)` in a thread → **N episodes concurrently**.
- `__aexit__` ends the session when the episode returns: one teardown path.
- `SANDBOX=agentcore-runtime` → your own Runtime: the agent loop runs **in the session** and dials the trainer back, which the Code Interpreter can't.

© 2026, Amazon Web Services, Inc. or its affiliates.

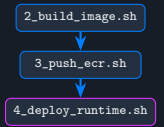
Deploying a Runtime takes three steps, and only the dial-back path needs ingress

Build, IAM, wire

1. **Build + deploy:** image (**arm64** microVM / **amd64** Instance) → ECR → `CreateAgentRuntime`. The image serves `sandbox_server_agent.py`: `write_file` / `exec` / `read_file`.
2. **IAM:** an execution role (ECR pull, logs) + an operator role with `InvokeAgentRuntime`. Instances also need `CreateCapacityProvider`.
3. **Wire** (`deploy/runtime.env`): `AGENTCORE_SANDBOX_RUNTIME_ARN` · `..._ARNS` (round-robin) · `..._ARN_BY_IMAGE` (per image).

Networking: pull, don't expose

SigV4 invoke, **no inbound port**. Only dial-back (in-session harness → trainer) needs ingress: one SG→SG rule under `networkMode=VPC`, plus interface endpoints (`ecr.api`, `ecr.dkr`, `s3`, `logs`, `sts`, `bedrock-agentcore`); a missing one fails create with a **null reason**.

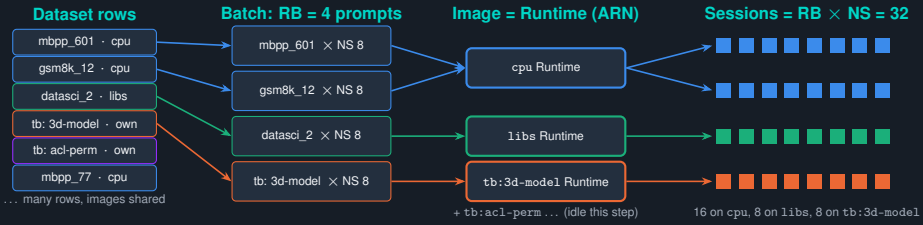


Each demo wraps the steps as these scripts.

© 2026, Amazon Web Services, Inc. or its affiliates.

Every row names a task and an image; each image is one Runtime; RB × NS sets sessions per step

Figure 3 CPU tasks + Terminal-Bench in one batch (`DATASET=mbpp,terminalbench`): row → Runtime → sessions



- **Row = task + image.** Rows may share one: every MBPP / GSM8K row → `cpu`; pandas / scipy / sklearn → `libs`; each Terminal-Bench task → its own image. The NS samples of a prompt share it.
- **Image = one Runtime**, via the map image → ARN. An unmapped image is an **error, never a fallback**. `cpu` + `libs` = 2 Runtimes; Terminal-Bench adds one per task (166 usable of 218 deployed).
- **Sessions per step = RB × NS** (default 4 × 8 = **32**), one fresh session each, spread over the Runtimes the batch touches.

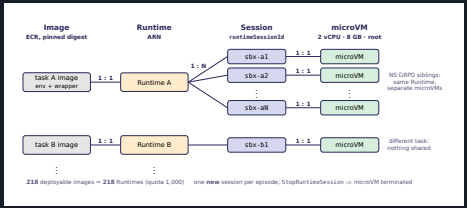
Source: `code/demos/demo_cpu_tasks` (row schema, `image_routing.py`) and the Terminal-Bench pool map (`tasks/arn_by_image.json`), 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Of 229 Terminal-Bench tasks, 166 are usable, each on its own image and Runtime

- **~229 terminal / SWE tasks, each its own environment**: its own OS, packages and a deliberately broken state. Reward = 1 only if every hidden `pytest` passes.
- The environment *is* the task → **one arm64 image + one Runtime per task**, routed by task id.
- Tasks mutate the whole OS (apt, `/etc`, services) → **a fresh microVM per episode**; the session is the isolation boundary.

Figure 4 **Image 1:1 Runtime 1:N session 1:1 microVM**



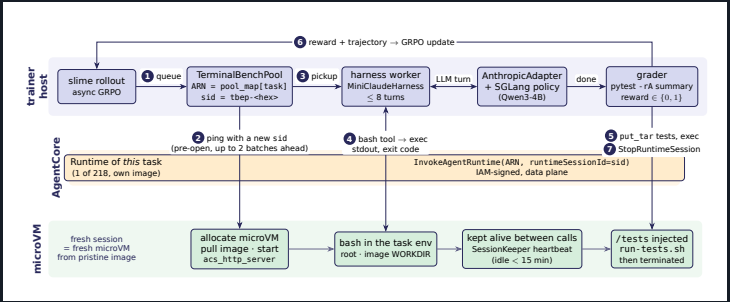
229 tasks → **218 deployable** (arm64, ≤ 2 GB) → **166 oracle-usable** → **135 train / 31 eval.**

Source: Terminal-Bench task audit in `code/demos/demo_cpu_tasks/terminalbench` (`tasks/arn_by_image.json`); project deck, 2026-09-25.

© 2026, Amazon Web Services, Inc. or its affiliates.

Every episode boots a fresh microVM: the first invoke with a new session id is the boot

Figure 5 One Terminal-Bench episode: route → fresh session → agent loop → grade → stop



- **Route → fresh session → agent loop → grade → stop.** The first invoke with a new `runtimeSessionId` **is** the boot.
- Hidden tests are injected only **after** the agent finishes; `StopRuntimeSession` terminates the microVM.
- One step = RB 4 × NS 8 = **32 episodes = 32 fresh microVMs**.

Source: `demo_cpu_tasks DATASET=terminalbench` configuration (RB 4, NS 8); project deck, 2026-09-25.

© 2026, Amazon Web Services, Inc. or its affiliates.

Serial cold boots alone would cost about 8 minutes per step, and slime cannot hide them

Table 5 Terminal-Bench key challenges

Challenge	Why it hurts
Cold start every episode	≈ 15 s for a 447 MB image; 32 per step ≈ **8 min** of serial boot if on the critical path
Bursty opens	all 8 GRPO siblings of a task start at once; account cap **25 new sessions/s**
slime can't hide it	only `CONC` × `engines` = 8 episodes visible; step N+1 drawn only after step N; slime knows nothing about sandboxes
Silent idle reclaim	a session idle ≈ 15 min is reclaimed (same id, empty sandbox), and weight sync can be that long
Broken / oversized tasks	7 don't build on arm64, 4 exceed the 2 GB cap, 47 fail their own reference solution
Runtime cap	1000 Runtimes per account → at most ~1000 images

Source: Terminal-Bench task audit and first AWS smoke (2026-09-23); AgentCore Service Quotas (2026-09-26).

© 2026, Amazon Web Services, Inc. or its affiliates.

Each challenge has a fix: fresh VMs, strict routing, oracle checks, pre-open, heartbeat, retries

Table 6 Terminal-Bench challenges and our solutions

Challenge	Solution
Isolation vs reuse	**Fresh microVM per episode**; reuse + reset rejected (OS state leaks into the reward)
Many images	**One image + Runtime per task**, strict routing (`tasks/arn_by_image.json`; unmapped = error)
Broken tasks	**Oracle check**: untouched env must FAIL, reference must PASS $\rightarrow$ **166 usable**
Cold start	**Look-ahead pre-open**: boot the next 2 batches' sessions before slime asks (slide 16)
Idle reclaim	**Heartbeat** (`SessionKeeper`) pings every open session (slide 17)
Infra failures	**Retry on a new id** ($3\times$); logged as `infra_error`, never scored as a wrong answer

Source: `code/demos/demo_cpu_tasks/terminalbench`; project deck, 2026-09-25.

© 2026, Amazon Web Services, Inc. or its affiliates.

Pre-opening cut the p50 open wait to $\leq$ 5 ms and the makespan by 15–20 %

1. **Look-ahead data source** (`tb_lookahead_source.py`, a `-data-source-path` plugin) replays slime's deterministic prompt order → the next 2 rollouts' prompts. **No slime change.**
2. **Bounded window:** `TB_PREOPEN_BATCHES=2` → **P = 64** sessions booted ahead, pooled per task; each claim starts the next boot. 1 → 32, 0 → off.
3. **Sizing:** boot hidden when P · E / W $\geq$ boot time; beyond that is idle cost. Tune by `open_wait_ms` ($\gg$ 0 → raise) vs `preopen_idle_ms` (the price).

Table 7 **Warm-start probe (24 episodes, batches of 6): cold open vs pre-open**

Backend	Mode	Wait p50	Waited	Makespan
own Runtime	cold → **preopen**	774 → **5 ms**	24 → 8 /24	31.9 → 25.5 s (−**20 %**)
Code Interpreter	cold → **preopen**	1715 → **0 ms**	24 → 6 /24	28.7 → 24.3 s (−**15 %**)

Source: Warm-start probe `demo_cpu_tasks/terminalbench/warmstart/logs/warmstart_20260925_100103.*` (2026-09-25, us-west-2).

© 2026, Amazon Web Services, Inc. or its affiliates.

With heartbeat on, own-Runtime sessions survived a long idle 3/3; without it, 0/3

- During weight sync no op reaches the sandbox → your own Runtime **silently reclaims** idle sessions (same id, empty sandbox, no error).
- `SessionKeeper` pings any session idle past `fraction × timeout` with `exec("true")`.
- **On by default** for own Runtimes (`HEARTBEAT=0` = off), idle timeout 3600 s as backstop. Code Interpreter: set a long session timeout instead.

Table 8 **Heartbeat vs silent idle reclaim**

Backend	Heartbeat	Survived 150 s idle (60 s timeout)	Resume
own Runtime	on	**3/3**	104 ms, files intact
own Runtime	off	0/3	**silently reclaimed**
Code Interpreter	on / off	0/3	expires at `sessionTimeoutSeconds`

Source: Warm-start probe `warmstart_20260925_100103` (2026-09-25, us-west-2), heartbeat leg.

© 2026, Amazon Web Services, Inc. or its affiliates.

Pre-open fully hid the boot (p95 wait 0), but solves are sparse at 4 of 382

Table 9 **Terminal-Bench smoke results (2026-09-23)**

Metric	Value
Episodes / infra errors	382, each on its own fresh microVM / **0**
Boot on the critical path (`open_wait_ms`)	**p50 0** · **p95 0**: pre-open hides it fully
Session open	p50 0.66 s · **100 % warm hits** · 0 retries
Pre-opened VM idle p50	**7.3 min** for a 21 s episode → ≈ 95 % idle → try `TB_PREOPEN_BATCHES=1`
Solves	**4** / **382** train, eval 0.0; 42 % pass ≥ 1 hidden test → sparse; next: curriculum / partial credit

In plain English: the sandbox side works: no infra errors and no visible boot wait. The open problem is the reward signal, not AgentCore.

Source: First Terminal-Bench AWS smoke, `demo_cpu_tasks DATASET=terminalbench` (2026-09-23).

© 2026, Amazon Web Services, Inc. or its affiliates.

The harness must scale with inference and the sandbox, or it silently becomes the ceiling

- **Use case covered:** scale an agentic-RL rollout by sizing **inference, harness and sandbox independently**; the decoupled harness tier (`harness_pool.py`) is the **default in every demo**, and `SCALE=1` turns all three up together (4 : 256 : 256, DeepScaleR on a stdlib image; `demo_cpu_scale_lanes` was retired into this preset, 2026-09-28).
- **Why this demo:** (1) the harness must scale **along with** SGLang and the sandbox, or a few coroutines silently become the ceiling; (2) it should scale **with the sandbox, not as part of the trainer**: the agent loop runs in the episode's session, as in DeepSeek DSec.
- **Design considerations:** **throughput = min(inference, harness, sandbox)**; each tier is its own pool, joined per episode (slide 20). Pinned together (worker = engine = microVM), the slowest pin sets the pace.

Source: `code/demos/demo_cpu_tasks` (`SCALE=1` preset), checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Each tier binds on a different limit, and each has its own signal to read

Table 10 The three tiers to scale

Tier	Scales with	Limited by	Signal that it binds
Inference (SGLang)	GPUs / engines	decode throughput	`engine_inflight`
Harness (agent loops)	Python concurrency	event loop, client pools	`qdepth`
Sandbox (AgentCore)	sessions in flight	**25 new/s**, 5000 concurrent, cost	`sandbox_wait_ms`

In plain English: the rollout goes only as fast as its slowest tier. Find which one is full before buying more of anything.

Source: `demo_cpu_tasks` harness tier (`harness_pool.py`); AgentCore Service Quotas (2026-09-26).

© 2026, Amazon Web Services, Inc. or its affiliates.

SCALE=1 runs 4 engines : 256 dispatchers : 256 sessions, opening at 20/s under the 25/s quota

Defaults in every demo: **W = RB × NS** dispatchers (32), a fresh session per episode, least-loaded engine. `SCALE=1` values below; client pools sized to 256 (boto3 default 10, **a silent cap**). Shrink one tier to make it the ceiling, then read its signal.

Table 11 **Tier sizing in** `demo_cpu_tasks SCALE=1`

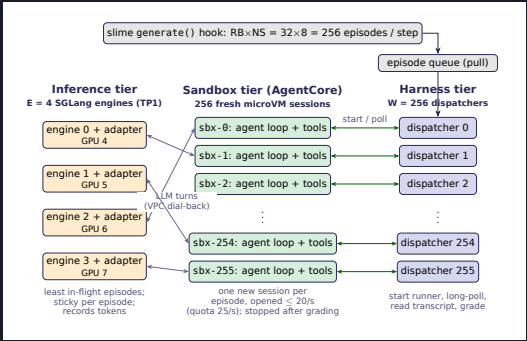
Tier	Default	Knob	Why this number
SGLang	**4 engines**, TP1, GPUs 4–7	`GPUS_PER_ENGINE` (1; default 2)	Qwen3-4B fits one L40S; other 4 GPUs train
Harness	**256** dispatchers	`HARNESS_WORKERS` (256)	= one step's batch, RB 32 × NS 8 = 256; CONC = 64 per engine
Sandbox	**256** fresh sessions in flight	`SANDBOX_SESSIONS` (0 = fresh per episode; N = N pooled)	one per in-flight episode
Open rate	**20 /s**	`SANDBOX_OPEN_RATE`	under the 25 /s quota → 256 open in ≈ 13 s

Source: `code/demos/demo_cpu_tasks SCALE=1` preset (2026-09-28); AgentCore Service Quotas (2026-09-26).

© 2026, Amazon Web Services, Inc. or its affiliates.

Nothing is pinned: each episode gets a fresh session and the least-loaded engine

Figure 6 Harness tier at `SCALE=1`: 4 engines : 256 dispatchers : 256 fresh sessions, agent loop in the session



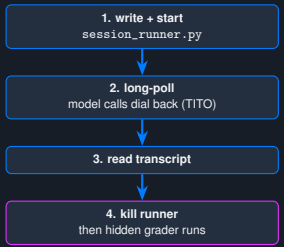
Per episode: a free dispatcher pulls it, opens a **fresh session** (sandbox first), starts the agent loop **inside** it; the loop dials back to the **least-loaded engine** (sticky for its turns). Nothing is pinned: this replaces the old hash pinning and per-engine workers.

Source: `demo_cpu_tasks` harness tier, every demo; shown at `SCALE=1` (2026-09-28).

© 2026, Amazon Web Services, Inc. or its affiliates.

The agent loop runs inside the sandbox session, so the harness tier scales with the sandbox

- **One agent loop per sandbox, in the same session**, in every demo, like DSec (only exception: `demo_cpu_branching` runs the real Claude Code CLI in its microVM).
- **Per episode:** write + start `session_runner.py` → long-poll → read transcript → **kill the runner, then the hidden grader** runs (the loop can't touch its reward).
- **Model calls dial back** (VPC, SG→SG) to the engine's `AnthropicAdapter`, which records the exact tokens (TITO).
- **So the harness tier scales with the sandbox tier**: trainer-side dispatchers only start and poll.
- **No fallback:** no `python3` or no path back → `HarnessPlacementError`; the loop never moves into the trainer.



© 2026, Amazon Web Services, Inc. or its affiliates.

One command runs SCALE=1; isolation defaults to a fresh microVM per episode

```
cd code/demos/demo_cpu_tasks
python3 harness_scaling_demo.py # offline min() sweep, seconds
SCALE=1 bash 5_run.sh # after steps 1-4: DATASET=deepscaler RB=32 NS=8
                                    # E=4 (TP1) HARNESS_WORKERS=256 CONC=64 -> S=256
```

- **Isolation:** default = **fresh microVM per episode**, stopped after grading.
- `SANDBOX_SESSIONS=N` (opt-in) pools N microVMs per image with a fresh `ep-<hex>` workdir only (`/tmp`, packages not reset → fine for DeepScaleR, not for OS-mutating tasks).

Source: `code/demos/demo_cpu_tasks` (`5_run.sh`, `harness_scaling_demo.py`); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

The 256-worker smoke ran 592 episodes and raw reward rose from 0.31 to 0.37

Table 12 Harness-scaling results

Run	Result
Session mode, live	verified per episode: good solution **1.0**, cheat **0.0**
Live pool check (64 : 64, 192 episodes)	**1280 ops ok**; engines 48 / 48 / 47 / 49
256-worker RL smoke (`SCALE=1`)	**592 episodes**, 2 steps × 256; `raw_reward` **0.31** → **0.37**, eval/deepscaler **0.27**
Earlier 4 × 4 × 4 lanes	eval **0.358**

In plain English: at 256 concurrent episodes the load spread evenly over the 4 engines (48 / 48 / 47 / 49), and raw reward rose over the 2 steps.

Source: 256-worker RL smoke, logs in `code/demos/demo_cpu_tasks/logs/scale/` (2026-09-28); live pool check and lanes run from the project deck (2026-09-25).

© 2026, Amazon Web Services, Inc. or its affiliates.

Five demo folders share the same hook, harness tier and step scripts, across microVMs, Instances, GPUs and EKS

3 main folders + an EKS twin + one experimental, each standalone in `code/demos/` with the same `1_prepare_data.sh` … `6_cleanup.sh` steps and the same harness tier.

Table 13 **Demo folders in the codebase**

Folder	What it shows
`demo_cpu_tasks`	literature tasks (MBPP, GSM8K, MATH-500, DeepScaleR, HumanEval, SQL, HotpotQA, xLAM, MCP, mixes) on a generic stdlib Runtime, plus **Terminal-Bench** (`DATASET=terminalbench`, one image + Runtime per task, slide 12; was `demo_cpu_tasks DATASET=terminalbench`); `SCALE=1` is the harness-scaling run (slide 19)
`demo_cpu_swebench`	one image + Runtime per **SWE-bench Verified** instance on **m5 Instances** (images $>$ 2 GB microVM cap); reuses the Terminal-Bench pool
`demo_cpu_swebench_eks`	the same SWE-bench workload on **EKS**, one fresh **gVisor** pod per episode; cluster not created yet
`demo_gpu_farm`	**KernelBench** CUDA RL; **one GPU instance serving many sessions** via a Ray farm (`FARM=fractional` 0.5 + 0.5 or `whole`), local or on a g6 (`LOCAL_GPU=0`); trusted-only
`demo_cpu_branching` *(experimental)*	the **real Claude Code CLI** as harness; one episode fans out into many samples

© 2026, Amazon Web Services, Inc. or its affiliates.

DSec outscales AgentCore's 25 opens/s; pre-open and heartbeat fill the gaps

Figure 7 DeepSeek DSec vs AgentCore as an RL sandbox

	DSec (DeepSeek)	AgentCore (AWS)
Scale		
Creation rate	$>$ 5,000 sandboxes /s	**25** /s new Runtime sessions
Running at once	380K	**5,000** Runtime · **1,000** Code Interpreter sessions
Environments	11K bases $\times$ 100K workspaces, layered at start time	one image per Runtime; max **1,000** Runtimes per account
Images and start time		
Image delivery	lazy loading: only the 4–13 % of the image a sandbox touches is fetched	**full image pulled per cold boot**: $\approx$ 0.8 s stdlib, $\approx$ 15 s at 447 MB, grows with size; **2 GB cap** on microVMs. **Ours:** no image on Code Interpreter (pandas/scipy/sklearn/torch-cpu preinstalled); install heavy deps at boot to fit 2 GB; $>$ 2 GB on a Runtime Instance (60–80 s m5, $\approx$ 207 s g6 boot, paid once via warm-shared session)
Hiding the cold boot	not needed: lazy loading keeps pull time low; pre-warming rejected (the bottleneck is I/O volume, not latency)	**pre-open (ours)**: while the current batch runs, the next 1–2 batches' sandboxes are opened (image pulled, microVM booted); ready-wait p50 774 ms $\rightarrow$ **5 ms**, makespan -20 %
Lifecycle		
Sandbox lifetime	p50 15–17 min, p99 $>$ 3 h	**15-min idle timeout** by default; max 8 h (microVM), 14 days (Runtime Instance)
Trainer stall	pause/resume, state kept	**no pause/resume, no state kept**; **heartbeat (ours)** keeps the sandbox alive
Architecture and safety		
Agent loop	outside the GPU training job	**outside the trainer too (ours)**: harness + tools in the same Runtime session, one loop per sandbox (`HARNESS_WHERE=session`, default on own Runtimes)
Reward hacking	AppArmor + per-task network allowlists	microVM boundary; Landlock jail and no TCP on shared pools; VPC mode for the network

green adjustable (quota or per-Runtime setting) **orange** our mitigation **red** AgentCore limit / missing

- **Scale:** $>$ 5,000 sandboxes/s and 380K at once (DSec) vs **25 /s** and 5,000 Runtime sessions (AgentCore).
- **Images:** lazy loading vs a full pull per cold boot; ours hides it with **pre-open**.
- **Stalls:** pause/resume vs none; ours bridges it with **heartbeat**.
- **Agent loop:** outside the trainer in both.

Green = adjustable, orange = our mitigation, red = AgentCore limit / missing. Full-size figure: slide 32.

Source: DSec: arXiv 2609.22978 (V3.2 $\rightarrow$ V4.1, 3M sandboxes/day). AgentCore: Service Quotas, this account, us-west-2, 2026-09-26.

© 2026, Amazon Web Services, Inc. or its affiliates.

CONCLUSION

DSec's sandbox lifetimes match our idle timeout, so heartbeat and lifetime metrics are now default

- **Their median sandbox lives about as long as our idle timeout** → heartbeat on by default; idle timeout 3600 s.
- **New metrics, every demo:** `perf/sandbox_ready_span_s`, `perf/sandbox_open_peak_per_s` (vs 25/s), per-session lifetime + longest idle gap (`<log>.sessions.csv`).
- **Terminal-Bench, re-parsed:** 414 sessions, lifetime p50 **471 s** / p95 **1841 s**; 55 outlived 900 s (kept by heartbeat); peak **36 opens/s**, **0 throttles**.
- **Not exposed by AgentCore:** image layers, session pause, thousands of creates/s.
- **Copied:** agent loop in the sandbox session (slide 23). **Still to copy:** per-task network allowlist.

Source: Terminal-Bench smoke (2026-09-23) re-parsed with the new session metrics; DSec arXiv 2609.22978; report Ch. 7.

© 2026, Amazon Web Services, Inc. or its affiliates.

Start generic, add images only when needed, and scale each tier on its own signal

- **Start on one generic stdlib Runtime**: most literature tasks need no custom image (the Code Interpreter has the packages, but can't host the in-session agent loop).
- **Add an image only when the environment differs**; route by `task.image`, AgentCore sees only an image and a session.
- **Hide boots with pre-open, bridge stalls with heartbeat** on your own Runtimes (slides 16–17).
- **Scale the tiers independently**: the harness tier with the sandbox, not in the trainer; diagnose the binding tier (`qdepth` / `sandbox_wait_ms` / `engine_inflight`) before adding capacity.
- **Isolation = the session**: fresh microVM when tasks mutate the OS; pooled microVM + fresh workdir for workdir-only tasks.

Sources for every number: slide 31.

© 2026, Amazon Web Services, Inc. or its affiliates.

Appendix

References · DSec comparison, full size

References: every number traces to a project run, log or document

Table 14 Sources used in this deck

Source	Used for
`docs/slides/agentcore_slime_deck.md` (2026-09-25)	source deck for all content
`docs/report/agentcore_slime_report.md`	launch costs, probe details, DSec comparison, GPU sweep (2026-09-21)
`demo_cpu_tasks/terminalbench/warmstart/logs/`	pre-open and heartbeat probe `warmstart_20260925_100103.*` (2026-09-25, us-west-2)
`code/demos/demo_cpu_tasks/terminalbench`	task audit (229 → 166), first AWS smoke (2026-09-23), `tb_lookahead_source.py`, `tasks/arn_by_image.json`
`code/demos/demo_cpu_tasks/logs/scale/`	256-worker SCALE=1 RL smoke (2026-09-28)
`code/demos/demo_cpu_tasks`	`harness_pool.py`, `session_runner.py`, `harness_scaling_demo.py`
`docs/agentcore-networking.md`	pull-vs-dial-back networking, VPC endpoints
AgentCore Service Quotas, us-west-2 (2026-09-26)	session rate, concurrency, Runtime and image limits
DSec, arXiv 2609.22978	DeepSeek sandbox figures (V3.2 → V4.1, 3M sandboxes/day)

© 2026, Amazon Web Services, Inc. or its affiliates.

Backup: AgentCore trails DSec on scale and images; loop placement matches

	DSec (DeepSeek)	AgentCore (AWS)
Scale		
Creation rate	> 5,000 sandboxes /s	**25 /s** new Runtime sessions
Running at once	380K	**5,000** Runtime · **1,000** Code Interpreter sessions
Environments	11K bases × 100K workspaces, layered at start time	one image per Runtime; max **1,000** Runtimes per account
Images and start time		
Image delivery	lazy loading: only the 4–13 % of the image a sandbox touches is fetched	**full image pulled per cold boot**: ≈ 0.8 s stdlib, ≈ 15 s at 447 MB, grows with size; **2 GB cap** on microVMs. **Ours:** no image on Code Interpreter (pandas/scipy/sklearn/torch-cpu preinstalled); install heavy deps at boot to fit 2 GB; > 2 GB on a Runtime Instance (60–80 s m5, ≈ 207 s g5 boot, paid once via warm-shared session)
Hiding the cold boot	not needed: lazy loading keeps pull time low; pre-warming rejected (the bottleneck is I/O volume, not latency)	**pre-open (ours)**: while the current batch runs, the next 1–2 batches' sandboxes are opened (image pulled, microVM booted); ready-wait p50 774 ms → **5 ms**, makespan −20 %
Lifecycle		
Sandbox lifetime	p50 15–17 min, p99 > 3 h	**15-min idle timeout** by default; max 8 h (microVM), 14 days (Runtime Instance)
Trainer stall	pause/resume, state kept	**no pause/resume, no state kept**; **heartbeat (ours)** keeps the sandbox alive
Architecture and safety		
Agent loop	outside the GPU training job	**outside the trainer too (ours)**: harness + tools in the same Runtime session, one loop per sandbox (`HARNESS_WHERE=session`, default on own Runtimes)
Reward hacking	AppArmor + per-task network allowlists	microVM boundary; Landlock jail and no TCP on shared pools; VPC mode for the network

green adjustable (quota or per-Runtime setting) **orange** our mitigation **red** AgentCore limit / missing

Source: DSec: arXiv 2609.22978 (V3.2 ⟶ V4.1, 3M sandboxes/day). AgentCore: Service Quotas, this account, us-west-2, 2026-09-26.

© 2026, Amazon Web Services, Inc. or its affiliates.

Thank you

Questions?

Kaige Yang · Senior Applied Scientist, AWS